UNIT 1

Foundations of Deep Learning — Neurons, Activation Functions and Backpropagation

Comprehensive Question Bank with Detailed Solutions • 17 Questions

Question 1 | Structure of a Multilayer Perceptron

> *Explain the structure of a Multilayer Perceptron. Describe the role of weights, biases, hidden layers, and activation functions in learning a nonlinear relationship.*

1. Definition

A **Multilayer Perceptron (MLP)** is a feedforward artificial neural network composed of at least three layers of neurons — an input layer, one or more hidden layers, and an output layer. Every neuron in one layer is connected to every neuron in the next layer, which is why an MLP is also called a **fully connected** or **dense** network. Information travels in one direction only, from input to output, with no cycles or feedback loops; hence the term *feedforward*.

The MLP is the direct extension of the single-layer perceptron. The single-layer perceptron can only separate classes with a straight line (a hyperplane), whereas the MLP, by stacking layers and inserting nonlinear activation functions between them, can approximate arbitrarily complex nonlinear mappings.

2. Layer-by-layer structure

- **Input layer** — contains one neuron per input feature. These neurons perform no computation; they simply hold and distribute the feature values $x_1, x_2, \ldots, x_n$ to the first hidden layer. If a dataset has 784 pixel features, the input layer has 784 nodes.
- **Hidden layers** — the computational core of the network. Each hidden neuron computes a weighted sum of all incoming signals, adds a bias, and passes the result through a nonlinear activation function. A network with two or more hidden layers is called a **deep** neural network.
- **Output layer** — produces the final prediction. Its size and activation depend on the task: one neuron with sigmoid for binary classification, k neurons with softmax for k-class classification, and one (or more) neuron with a linear activation for regression.

3. Forward propagation equations

For a network with one hidden layer, let $W^{(1)}$ be the input-to-hidden weight matrix, $b^{(1)}$ the hidden bias vector, $W^{(2)}$ the hidden-to-output weight matrix and $b^{(2)}$ the output bias:

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad \text{(hidden pre-activation)}$$
$$a^{(1)} = f(z^{(1)}) \quad \text{(hidden activation, f nonlinear)}$$
$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad \text{(output pre-activation)}$$
$$\hat{y} = g(z^{(2)}) \quad \text{(final output)}$$

For a general layer l this generalises to $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ and $a^{(l)} = f(z^{(l)})$, with $a^{(0)} = x$. In scalar form, the j-th neuron of layer l computes $z_j = \Sigma_i\, w_{ji}\, a_i + b_j$.

4. Role of each component

Component	Role in learning a nonlinear relationship
Weights (w)	Weights are the **learnable strength of each connection**. A large positive weight means the corresponding input strongly excites the neuron; a large negative weight means it strongly inhibits it; a near-zero weight means the input is effectively ignored. Geometrically, the weight vector fixes the *orientation* of the decision hyperplane. During training, weights are the parameters that gradient descent adjusts, so all knowledge extracted from the data is ultimately stored in the weights.
Biases (b)	The bias is a learnable constant added to the weighted sum. It **shifts the activation function left or right**, i.e. it controls the threshold at which a neuron fires, independently of the inputs. Geometrically it fixes the *offset* of the hyperplane from the origin. Without a bias, every decision boundary would be forced to pass through the origin, severely restricting the class of functions the network can represent. The bias also allows the network to fit data whose mean is non-zero.
Hidden layers	Hidden layers create a **hierarchy of learned internal representations**. Each layer transforms the representation produced by the previous layer into a new feature space in which the problem becomes progressively easier — ultimately linearly separable at the output layer. Early layers learn simple, local patterns (edges, short-range correlations) and deeper layers compose them into abstract concepts (shapes, objects). Increasing width increases the number of features per level of abstraction; increasing depth increases the number of levels of abstraction, and depth is usually far more parameter-efficient than width.
Activation functions	Activation functions inject **nonlinearity**. Without them the entire network would collapse into a single linear transformation (see Question 3) and no amount of depth would help. They also bound or rectify the signal, which stabilises training, and their derivatives control how gradients flow backwards during backpropagation. The choice of activation therefore determines both what the network *can* represent and how easily it *can be trained*.

5. How the pieces combine to learn a nonlinear relationship

Each hidden neuron with a ReLU activation contributes one linear piece (a 'hinge') to the overall function; the output layer takes a weighted combination of these hinges. With enough hidden neurons the network builds a piecewise-linear surface that can approximate any continuous curve to arbitrary accuracy. This is the informal statement of the **Universal Approximation Theorem** (Cybenko, 1989; Hornik, 1991): a feedforward network with a single hidden layer containing a finite number of neurons and a non-constant, bounded, continuous activation function can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to any desired accuracy.

Learning proceeds by (i) forward propagation to obtain $\hat{y}$, (ii) computation of a loss $L(y, \hat{y})$, (iii) backpropagation of gradients through the layers, and (iv) a gradient-descent update $W \leftarrow W - \eta\, \partial L/\partial W$. Repeating this over many epochs progressively shapes the weights and biases so that the composite nonlinear mapping fits the training data.

6. Worked example of architecture size

Consider an MLP with 4 inputs, one hidden layer of 5 neurons and 3 output neurons:

```
Weights : 4×5 + 5×3 = 20 + 15 = 35
Biases  : 5 + 3           = 8
Total trainable parameters = 43
```

7. Advantages and limitations

- **Advantages:** universal function approximation, automatic feature learning (no hand-crafted features), applicable to classification and regression, and highly parallelisable on GPUs.
- **Limitations:** the fully connected structure produces a very large number of parameters for high-dimensional inputs such as images (which motivates CNNs), it ignores spatial or temporal structure, it requires large labelled datasets, it is prone to overfitting, and it behaves largely as a black box with limited interpretability.

Question 2 | Mathematical Model of an Artificial Neuron

> *Explain the mathematical model of an artificial neuron. Describe the role of weighted summation, bias, and activation function.*

1. Biological motivation

An artificial neuron is a simplified mathematical abstraction of a biological neuron. In biology, dendrites receive electrical signals from other neurons, the synapses attenuate or amplify each signal, the cell body (soma) integrates them, and the neuron fires an output along its axon only if the integrated signal exceeds a threshold. The artificial model maps these directly: **inputs ↔ dendrites, weights ↔ synaptic strengths, summation ↔ soma, activation function ↔ firing threshold, output ↔ axon**. The first such model was the McCulloch–Pitts neuron (1943), which used binary inputs and a hard threshold; the modern neuron generalises it to real-valued inputs, real-valued learnable weights, and a differentiable activation.

2. The mathematical model

Given an input vector $x = [x_1, x_2, \ldots, x_n]^T$, a weight vector $w = [w_1, w_2, \ldots, w_n]^T$ and a bias b, the neuron performs a two-stage computation.

Stage 1 — linear combination (pre-activation):

$$z = w_1x_1 + w_2x_2 + \ldots + w_n\ x_n + b$$
$$= \Sigma_{i=1}^{n}\ w_i x_i + b$$
$$= w^T x + b \qquad \text{(vector form)}$$

Stage 2 — nonlinear activation (post-activation):

$$a = f(z) = f(w^T x + b)$$

Here z is called the *net input*, *logit* or *pre-activation value*, and a is the neuron's *output* or *activation*. The pair (w, b) constitutes the **trainable parameters** of the neuron; the function f is a fixed design choice, not learned.

3. Role of weighted summation

- It performs **feature weighting**: each weight w_i encodes how important input x_i is for this particular neuron's decision, and the sign encodes whether the influence is excitatory (positive) or inhibitory (negative).
- It performs **dimensionality reduction**: an n-dimensional input is compressed into a single scalar z, so the neuron acts as a learned feature detector that responds most strongly when the input pattern aligns with its weight vector.
- It has a clean **geometric interpretation**: since $w^Tx + b = 0$ defines a hyperplane in $\mathbb{R}^n$, the weighted sum measures the signed distance of the input point from that hyperplane, scaled by $\|w\|$. The magnitude $\|w\|$ therefore controls how sharply the neuron discriminates near the boundary.
- It is the **only place where information from different inputs is mixed**; the activation function afterwards acts element-wise and cannot combine features.

4. Role of the bias

- The bias **shifts the activation function horizontally**, controlling the threshold at which the neuron becomes active. For a sigmoid, the neuron outputs 0.5 when $w^Tx = -b$, so b sets the firing point.
- It allows the decision boundary to be **offset from the origin**. Without b, the hyperplane $w^Tx = 0$ must pass through the origin, and a neuron could never, for example, learn the function 'output 1 for all inputs' or fit data whose classes are not centred at the origin.

- It is the neural-network analogue of the **intercept term** in linear regression y = mx + c, and it increases the model's flexibility at the cost of only one extra parameter per neuron.
- It can be absorbed into the weight vector by appending a constant input $x_0 = 1$ with weight $w_0 = b$, which is a convenient trick for compact matrix implementations.

5. Role of the activation function

- **Introduces nonlinearity**, without which any stack of neurons collapses into a single linear map (Question 3). This is the single most important role.
- **Bounds or rectifies the output**, which keeps activations in a numerically stable range and prevents them from exploding as they propagate through many layers.
- **Provides a probabilistic interpretation** when required — sigmoid maps $\mathbb{R} \to (0, 1)$ for a binary probability, softmax maps a vector to a probability distribution over classes.
- **Controls gradient flow**: backpropagation multiplies by f′(z) at every layer, so the shape of f directly determines whether gradients vanish, explode, or propagate cleanly.
- **Creates sparsity** in the case of ReLU, since roughly half of the neurons output exactly zero for a random input, which is both computationally efficient and a useful implicit regulariser.

6. Numerical illustration

Let $x = [2, -1, 3]^T$, $w = [0.5, 0.8, -0.2]^T$ and $b = 0.3$:

$$z = (0.5)(2) + (0.8)(-1) + (-0.2)(3) + 0.3$$
$$= 1.0 - 0.8 - 0.6 + 0.3 = -0.1$$
$$a = \sigma(-0.1) = 1 / (1 + e^{0.1}) = 1 / 2.1052 = 0.4750$$

The neuron is therefore slightly *inhibited*: its output lies just below the neutral value 0.5. Had the bias been b = 0.6 instead, z would be +0.2 and a = 0.5498, showing concretely how the bias alone can flip the neuron's decision.

Question 3 | Necessity of Nonlinear Activation Functions

> *Explain why nonlinear activation functions are necessary in a deep neural network. What would happen if every layer used only a linear activation function?*

1. The central claim

Nonlinear activation functions are what give a deep network its representational power. Without them, depth is mathematically meaningless: a network of any number of layers reduces exactly to a single linear layer. Nonlinearity is therefore not an optional enhancement but the defining ingredient that separates a deep neural network from linear regression.

2. Proof that stacked linear layers collapse

Suppose every layer uses the identity activation f(z) = z. For a two-layer network:

$$a^{(1)} = W^{(1)}x + b^{(1)}$$
$$\hat{y} = W^{(2)}a^{(1)} + b^{(2)}$$
$$= W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}$$
$$= (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)})$$

Now define $W' = W^{(2)}W^{(1)}$ and $b' = W^{(2)}b^{(1)} + b^{(2)}$. Then:

$$\hat{y} = W'x + b' \quad \text{— a single linear layer}$$

By induction the same argument extends to L layers: the composition $W^{(L)}W^{(L-1)}\ldots W^{(1)}$ is itself just one matrix, and the accumulated bias terms form one vector. Formally, **the composition of affine maps is affine**. A 100-layer linear network has exactly the same expressive power as a 1-layer linear network — it is simply an extremely inefficient and badly conditioned way of parameterising a single matrix.

3. Consequences of an all-linear network

Consequence	Explanation
Depth becomes useless	Adding layers cannot increase the set of functions the network can represent. Hierarchical feature learning is impossible because every 'level' is just another linear rearrangement of the same information.
Only linear decision boundaries	The network can only produce a hyperplane. It is equivalent to linear or logistic regression and fails on any dataset that is not linearly separable.
The XOR problem is unsolvable	XOR requires two separating lines. A linear model cannot achieve better than 75% accuracy on the four XOR points, no matter how deep it is or how long it is trained.
Wasted parameters and unstable optimisation	Millions of parameters are used to express a function with only n×m + m degrees of freedom. The parameterisation is heavily redundant, the loss surface has flat and degenerate directions, and the product of many matrices tends to explode or vanish numerically.
No approximation guarantee	The Universal Approximation Theorem explicitly requires a non-polynomial (hence nonlinear) activation. With linear activations the theorem does not apply and complex real-world mappings — images, speech, language — cannot be modelled.

4. What nonlinearity buys us

- **Curved and piecewise-linear decision boundaries.** Each ReLU unit contributes one hinge; combining many hinges lets the network carve the input space into a large number of linear regions, approximating any smooth boundary.

- **Hierarchical feature composition.** Because each layer's output is a genuinely new, non-affine function of its input, deeper layers can detect increasingly abstract patterns rather than re-scaled copies of the same ones.
- **Exponential efficiency of depth.** The number of linear regions a ReLU network can express grows *exponentially* with depth but only polynomially with width. Depth is therefore a far cheaper way to buy expressive power — but this advantage exists only when the layers are nonlinear.
- **Universal approximation.** With a nonlinear squashing activation, a single sufficiently wide hidden layer can approximate any continuous function on a compact domain to arbitrary precision.

5. Important qualification

Nonlinearity is required in the **hidden** layers. The **output** layer is frequently linear by design: for a regression task the output neuron uses the identity activation so that predictions can take any real value. Similarly, linear operations such as pooling averages, skip connections and 1×1 convolutions are useful precisely because nonlinearity is supplied elsewhere. The rule is therefore: *at least one nonlinearity between every pair of weight matrices whose composition should not be collapsible.*

Question 4 | Sigmoid vs. Tanh vs. ReLU

> *Distinguish Sigmoid, Tanh, and ReLU activation functions with respect to output range, advantages, limitations, and suitable use in deep neural networks.*

1. Definitions and derivatives

```
Sigmoid : σ(z) = 1 / (1 + e^(−z))          σ′(z) = σ(z)·(1 − σ(z))
Tanh    : tanh(z) = (e^z − e^(−z))/(e^z + e^(−z))    tanh′(z) = 1 − tanh²(z)
ReLU    : ReLU(z) = max(0, z)              ReLU′(z) = 1 if z>0, 0 if z<0
```

A useful identity linking the first two is $\tanh(z) = 2\sigma(2z) - 1$, which shows that tanh is simply a rescaled and re-centred sigmoid.

2. Comparison table

Property	Sigmoid	Tanh	ReLU
Output range	(0, 1)	(−1, 1)	[0, ∞)
Zero-centred?	No — always positive	Yes	No — always non-negative
Max derivative	0.25 (at z = 0)	1.0 (at z = 0)	1 (for all z > 0)
Saturation	Saturates at both ends; gradient ≈ 0 for |z| > 5	Saturates at both ends, but less severely than sigmoid	Never saturates for z > 0; completely dead for z < 0
Computational cost	High — requires exp()	High — requires exp()	Very low — a single comparison
Vanishing gradient	Severe. Since σ′ ≤ 0.25, gradients shrink by at least 4× per layer	Present but milder, since tanh′ ≤ 1	Largely avoided in the active region, where the gradient is exactly 1
Convergence speed	Slowest	Faster than sigmoid (zero-centred output removes the zig-zag update problem)	Fastest — empirically several times faster than tanh
Main advantage	Smooth, differentiable, gives a direct probability interpretation	Zero-centred and stronger gradients, so hidden-layer learning is better behaved	Cheap, sparse, non-saturating; makes very deep networks trainable
Main limitation	Vanishing gradients, non-zero-centred output, expensive	Still saturates, still expensive	**Dying ReLU** — a neuron stuck with z < 0 outputs 0 and receives zero gradient forever; also non-differentiable at z = 0 and unbounded above
Recommended use	Output layer for **binary classification** and for gating in LSTM/GRU	Hidden layers of shallow networks, RNNs, and where zero-centred outputs matter	**Default choice for hidden layers** of deep networks, CNNs, and MLPs

3. Discussion of the key differences

Why zero-centring matters. Because sigmoid outputs are always positive, every weight feeding into a neuron receives a gradient of the same sign in a given step, so the weight vector can only move in a restricted set of directions — producing a characteristic slow zig-zag trajectory. Tanh, being symmetric about the origin, removes this effect and generally converges faster.

Why ReLU dominates in practice. For $z > 0$ the derivative is exactly 1, so gradients pass backwards through many layers without shrinking, which is what made networks with dozens of layers trainable. ReLU also produces genuine sparsity — for a randomly initialised layer roughly half the units output exactly zero — which reduces computation and acts as a mild regulariser.

The dying ReLU problem and its fixes. If a large gradient update drives a neuron's weights such that $z < 0$ for every training example, the neuron outputs 0 permanently and its gradient is 0, so it can never recover. Variants address this:

```
Leaky ReLU : f(z) = z if z>0, else αz     (α ≈ 0.01, fixed)
PReLU      : same as above but α is learned
ELU        : f(z) = z if z>0, else α(e^z – 1)
GELU/Swish : smooth, self-gated; used in Transformers
```

4. Practical selection rule

- Hidden layers of a deep feedforward or convolutional network → **ReLU** (switch to Leaky ReLU / ELU if you observe many dead units).
- Output layer for binary classification → **sigmoid** with binary cross-entropy loss.
- Output layer for multi-class classification → **softmax** with categorical cross-entropy.
- Output layer for regression → **linear** (no activation).
- Recurrent networks and gating mechanisms → **tanh** for state, **sigmoid** for gates.

Question 5 | The Vanishing Gradient Problem

> *Explain the vanishing gradient problem in deep neural networks. Discuss how activation functions and increasing network depth can contribute to this problem.*

1. Statement of the problem

The **vanishing gradient problem** occurs when the error gradients computed by backpropagation become exponentially smaller as they are propagated from the output layer back towards the input layer. The weights of the early layers then receive updates so tiny that they are effectively frozen at their random initial values, while only the last few layers actually learn. The network trains slowly, plateaus at a poor loss, and — paradoxically — performs worse as more layers are added. The problem was identified by Sepp Hochreiter in 1991 and was the principal reason deep networks were considered untrainable before roughly 2010.

2. Mathematical origin — the chain rule as a product

Backpropagation computes the gradient with respect to an early weight as a **product** of per-layer terms. For an L-layer network the gradient at layer 1 is:

$$\partial L/\partial W^{(1)} = \partial L/\partial a^{(L)} \cdot \Pi_{l=2}^{L} \left[f'(z^{(l)}) \cdot W^{(l)} \right] \cdot f'(z^{(1)}) \cdot x^{T}$$

The gradient is thus a product of L factors. If the typical magnitude of each factor is r, the gradient at the first layer scales as r^L. Three regimes follow:

- $r < 1 \rightarrow$ the gradient **vanishes** exponentially with depth.
- $r \approx 1 \rightarrow$ the gradient is well conditioned and training is stable.
- $r > 1 \rightarrow$ the gradient **explodes** exponentially (the complementary problem).

3. Contribution of the activation function

The factor $f'(z^{(l)})$ is fixed entirely by the choice of activation:

Activation	Maximum of f'	Effect on the gradient product
Sigmoid	0.25	Even in the best case each layer multiplies the gradient by at most ¼. Over 10 layers the attenuation is at most $0.25^{10} \approx 9.5 \times 10^{-7}$ — a millionfold reduction — and in the saturated region ($\lvert z \rvert > 5$) $f' \approx 0$ makes it far worse.
Tanh	1.0	Better, because the maximum derivative is 1 at z = 0. However tanh also saturates, so away from the origin f' falls rapidly towards 0 and the same exponential decay reappears.
ReLU	1.0 (for z > 0)	The derivative is *exactly* 1 for all positive pre-activations, so active paths pass the gradient backwards undiminished. This is the main reason ReLU enabled deep learning. The trade-off is the dying-ReLU problem, where f' = 0 kills the path entirely.

Saturation is the key mechanism. A sigmoid neuron with z = 6 outputs 0.9975 and has derivative 0.0025. Multiplying by 0.0025 at every layer annihilates the gradient within two or three layers. Large weights, unnormalised inputs, and poor initialisation all push pre-activations into this saturated regime.

4. Contribution of network depth

Depth is the exponent in the expression r^L, so it amplifies whatever per-layer attenuation exists. Assuming an average factor of 0.25 per layer:

$$\text{Depth } 5 : 0.25^{5} \approx 9.8 \times 10^{-4}$$

Depth 10 : $0.25^{10} \approx 9.5 \times 10^{-7}$

Depth 20 : $0.25^{20} \approx 9.1 \times 10^{-13}$

A gradient of order 10^{-13} multiplied by a learning rate of 0.01 gives an update of order 10^{-15}, which is below the resolution of 32-bit floating point. The first layers therefore never move. The problem is especially acute in recurrent networks, where the *same* weight matrix is applied at every time step, so a sequence of length 100 behaves like a 100-layer network.

5. Symptoms in practice

- Training loss falls rapidly for a few epochs and then plateaus at a high value.
- Weights and gradient norms in the early layers remain almost identical to their initial values while the last layers change substantially.
- Adding more layers *degrades* both training and validation accuracy — a degradation that is an optimisation failure, not overfitting.
- Histograms of hidden activations show most units pinned near 0 or 1 (saturated).

6. Remedies

Remedy	How it helps
ReLU and its variants	Derivative of exactly 1 in the active region removes the per-layer shrinkage. Leaky ReLU / ELU additionally keep a small gradient for negative inputs, avoiding dead units.
Proper weight initialisation	He initialisation (variance 2/n_in) for ReLU and Xavier/Glorot (variance 1/n_in) for tanh keep the variance of activations and gradients roughly constant across layers, so r stays near 1.
Batch Normalisation	Normalises each layer's pre-activations to zero mean and unit variance, keeping them in the high-gradient region of the activation function and greatly reducing sensitivity to initialisation.
Residual (skip) connections	The identity shortcut in a ResNet block, y = F(x) + x, gives the gradient an additive path with derivative 1, so it can bypass the multiplicative chain entirely. This is what made 100+ layer networks trainable.
Gated recurrent architectures	LSTM and GRU use additive cell-state updates and multiplicative gates that maintain a near-constant error flow over long sequences.
Better optimisers	Adam and RMSProp normalise each parameter's update by a running estimate of its gradient magnitude, so even very small gradients produce meaningful steps.
Gradient clipping	Addresses the opposite failure mode (exploding gradients) by rescaling the gradient when its norm exceeds a threshold.

Question 6 | Backpropagation in a Feedforward Neural Network

> *Explain the process of backpropagation in a feedforward neural network. Illustrate and derive the flow of input data, predicted output, loss, and gradients using a neat diagram.*

1. What backpropagation is

Backpropagation (backward propagation of errors) is an efficient algorithm for computing the gradient of the loss function with respect to every weight and bias in the network. It is not itself a learning algorithm — it is the gradient-computation engine that supplies gradients to an optimiser such as gradient descent. Its efficiency comes from applying the chain rule in reverse topological order and reusing intermediate results, so the entire gradient is obtained in a single backward sweep costing roughly the same as one forward pass, rather than the O(P) separate passes that naive numerical differentiation of P parameters would require.

2. The complete diagram

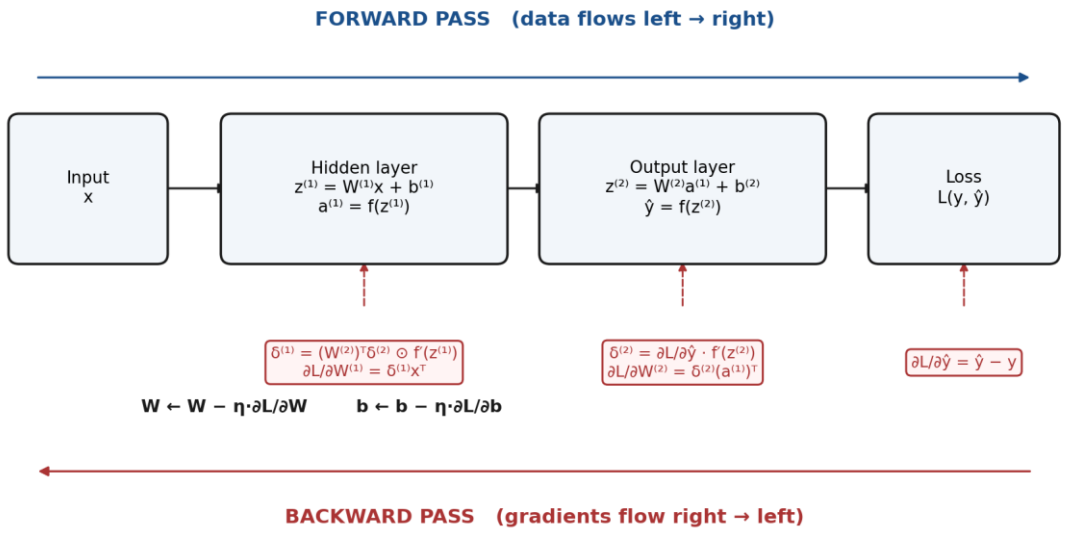


Figure 6.1 — Forward flow of data and backward flow of gradients in a two-layer feedforward network.

3. Step 1 — Forward pass

The input is propagated layer by layer to produce a prediction:

$$z^{(1)} = W^{(1)}x + b^{(1)} \qquad a^{(1)} = f(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \qquad \hat{y} = g(z^{(2)})$$

All intermediate values $z^{(l)}$ and $a^{(l)}$ are **cached**, because they are required again during the backward pass.

4. Step 2 — Loss computation

$$\text{Regression} : L = \tfrac{1}{2}(y - \hat{y})^2$$

$$\text{Binary classification} : L = -[y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y})]$$

The loss is a single scalar that measures how far the prediction is from the target. It is the quantity whose gradient we propagate.

5. Step 3 — Backward pass (derivation)

(a) Gradient at the output. For the squared-error loss:

$$\partial L/\partial \hat{y} = -(y - \hat{y}) = \hat{y} - y$$

(b) Output-layer error term. Applying the chain rule through the output activation g:

$$\delta^{(2)} = \partial L/\partial z^{(2)} = (\partial L/\partial \hat{y}) \cdot g'(z^{(2)}) = (\hat{y} - y) \cdot g'(z^{(2)})$$

For a sigmoid output, $g'(z^{(2)}) = \hat{y}(1 - \hat{y})$, so $\delta^{(2)} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$.

(c) Output-layer parameter gradients. Since $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$, we have $\partial z^{(2)}/\partial W^{(2)} = a^{(1)}$ and $\partial z^{(2)}/\partial b^{(2)} = 1$:

$$\partial L/\partial W^{(2)} = \delta^{(2)} \, (a^{(1)})^{T}$$

$$\partial L/\partial b^{(2)} = \delta^{(2)}$$

(d) Propagating the error to the hidden layer. The hidden activation influences the loss only through $z^{(2)}$, so:

$$\partial L/\partial a^{(1)} = (W^{(2)})^{T} \, \delta^{(2)}$$

$$\delta^{(1)} = \partial L/\partial z^{(1)} = [(W^{(2)})^{T} \, \delta^{(2)}] \odot f'(z^{(1)})$$

where $\odot$ denotes element-wise (Hadamard) multiplication. This is the recursive heart of the algorithm: **the error of a layer equals the back-projected error of the next layer, gated by the local derivative of its own activation.**

(e) Hidden-layer parameter gradients.

$$\partial L/\partial W^{(1)} = \delta^{(1)} \, x^{T}$$

$$\partial L/\partial b^{(1)} = \delta^{(1)}$$

(f) General recursive form for any layer l:

$$\boldsymbol{\delta^{(l)} = [(W^{(l+1)})^{T} \, \delta^{(l+1)}] \odot f'(z^{(l)})}$$

$$\boldsymbol{\partial L/\partial W^{(l)} = \delta^{(l)} \, (a^{(l-1)})^{T} \qquad \partial L/\partial b^{(l)} = \delta^{(l)}}$$

6. Step 4 — Parameter update

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \partial L/\partial W^{(l)}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \cdot \partial L/\partial b^{(l)}$$

where η is the learning rate. The negative sign moves the parameters in the direction of steepest *descent* of the loss.

7. The complete algorithm

1. Initialise all weights randomly (He or Xavier) and all biases to zero.
2. For each epoch, for each mini-batch: perform the forward pass and cache $z^{(l)}$ and $a^{(l)}$.
3. Compute the loss $L(y, \hat{y})$.
4. Compute the output error $\delta^{(L)} = \partial L/\partial \hat{y} \odot g'(z^{(L)})$.
5. For $l = L-1$ down to 1, propagate $\delta^{(l)} = [(W^{(l+1)})^{T}\delta^{(l+1)}] \odot f'(z^{(l)})$.
6. Accumulate $\partial L/\partial W^{(l)}$ and $\partial L/\partial b^{(l)}$ for every layer.
7. Update all parameters with the optimiser.
8. Repeat until the validation loss stops improving.

8. Key observations

- The forward pass **caches** activations, which is why training consumes far more memory than inference.
- Gradients flow **backwards** while data flows **forwards** — the two passes traverse the same computational graph in opposite directions.
- Because δ is built up as a *product* of local derivatives, backpropagation is the direct mechanism behind the vanishing and exploding gradient problems of Question 5.

- Backpropagation requires every operation in the network to be differentiable (or at least sub-differentiable, as with ReLU at z = 0, where the convention f'(0) = 0 is used).

Question 7 | Numerical — Forward Pass through a 2-2-1 Network (ReLU + Sigmoid)

> *Consider a neural network having two input neurons, two hidden neurons, and one output neuron. The inputs are $x_1 = 1.0$ and $x_2 = 2.0$. The input-to-hidden weights are $w_{11} = 0.2$, $w_{12} = 0.4$, $w_{21} = 0.3$ and $w_{22} = -0.2$, where w_{ij} connects input x_i to hidden neuron h_j. Hidden biases are b_h1 = 0.1 and b_h2 = 0.2. Use ReLU activation. The hidden-to-output weights are $v_1 = 0.5$ and $v_2 = -0.4$, and output bias b_o = 0.1. Use sigmoid activation. (i) Illustrate the complete neural network architecture clearly showing weights and biases. (ii) Evaluate the outputs of h_1 and h_2. (iii) Evaluate the final network output.*

Given data

Inputs : $x_1 = 1.0$, $x_2 = 2.0$

Input→hidden : $w_{11} = 0.2$ ($x_1 \to h_1$), $w_{12} = 0.4$ ($x_1 \to h_2$)

$w_{21} = 0.3$ ($x_2 \to h_1$), $w_{22} = -0.2$ ($x_2 \to h_2$)

Hidden biases : b_h1 = 0.1, b_h2 = 0.2 Activation: ReLU

Hidden→output : $v_1 = 0.5$, $v_2 = -0.4$, b_o = 0.1 Activation: Sigmoid

(i) Network architecture

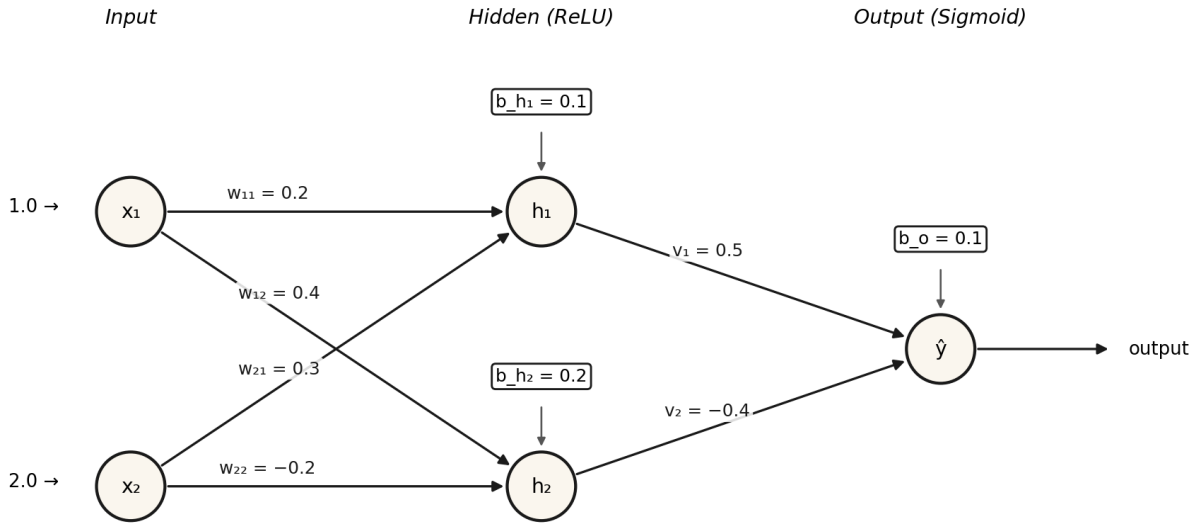


Figure 7.1 — Complete 2–2–1 architecture with all weights and biases labelled.

The network is fully connected with 2 × 2 = 4 input-to-hidden weights, 2 × 1 = 2 hidden-to-output weights, and 3 biases, giving 6 weights + 3 biases = **9 trainable parameters**.

(ii) Hidden layer outputs

Hidden neuron h_1 — collect every incoming signal (w_{11} from x_1 and w_{21} from x_2) and add the bias:

$$z_{h1} = x_1 \cdot w_{11} + x_2 \cdot w_{21} + b_{h1}$$
$$= (1.0)(0.2) + (2.0)(0.3) + 0.1$$
$$= 0.2 + 0.6 + 0.1$$
$$= 0.9$$
$$h_1 = \text{ReLU}(0.9) = \max(0, 0.9)$$
$$\mathbf{h_1 = 0.9}$$

Hidden neuron h_2 — incoming weights are w_{12} from x_1 and w_{22} from x_2:

$$z_h2 = x_1 \cdot w_{12} + x_2 \cdot w_{22} + b_h2$$
$$= (1.0)(0.4) + (2.0)(-0.2) + 0.2$$
$$= 0.4 - 0.4 + 0.2$$
$$= 0.2$$
$$h_2 = \text{ReLU}(0.2) = \max(0, 0.2)$$
$$\mathbf{h_2 = 0.2}$$

Both pre-activations are positive, so both ReLU units are in their active (linear) region and neither is dead. Had either z been negative, the corresponding hidden output would have been exactly 0.

(iii) Final network output

Output pre-activation:

$$z_o = h_1 \cdot v_1 + h_2 \cdot v_2 + b_o$$
$$= (0.9)(0.5) + (0.2)(-0.4) + 0.1$$
$$= 0.45 - 0.08 + 0.1$$
$$= 0.47$$

Sigmoid activation:

$$\hat{y} = \sigma(0.47) = 1 / (1 + e^{(-0.47)})$$
$$e^{(-0.47)} = 0.6250$$
$$\hat{y} = 1 / (1 + 0.6250) = 1 / 1.6250$$
$$\mathbf{\hat{y} = 0.6154 \quad (\approx 0.615)}$$

Summary of results

Quantity	Pre-activation (z)	Activation used	Output
Hidden neuron h_1	0.90	ReLU	**0.90**
Hidden neuron h_2	0.20	ReLU	**0.20**
Output neuron $\hat{y}$	0.47	Sigmoid	**0.6154**

Interpretation

The output 0.6154 lies above 0.5, so for a binary classification task with a threshold of 0.5 this sample would be classified as **class 1**, with a confidence of about 61.5%. The value is only moderately above 0.5 because the output pre-activation z_o = 0.47 is small — the positive contribution of h_1 (+0.45) is partly cancelled by the negative contribution of h_2 (−0.08). Note also that this is only a forward pass: no learning has occurred, and the prediction reflects the given (untrained) weights.

Question 8 | Numerical — Loss and Output Gradients for Binary Classification

> *For a binary classification problem, a neural network produces the predicted output $\hat{y} = 0.72$ for a training sample whose actual output is $y = 1$. The loss function is $L = \frac{1}{2}(y - \hat{y})^2$. Evaluate: (i) the value of the loss L; (ii) the gradient of the loss with respect to the predicted output $\partial L/\partial \hat{y}$; (iii) if the output neuron uses the sigmoid activation function, the gradient of the loss with respect to the neuron's pre-activation value z, using $\partial L/\partial z = (\partial L/\partial \hat{y}) \cdot \hat{y}(1 - \hat{y})$.*

Given data

```
Predicted output : ŷ = 0.72
Actual output    : y  = 1
Loss function    : L = ½(y − ŷ)²
Output activation: sigmoid, so σ'(z) = ŷ(1 − ŷ)
```

(i) Value of the loss L

$$L = \tfrac{1}{2}(y - \hat{y})^2$$
$$= \tfrac{1}{2}(1 - 0.72)^2$$
$$= \tfrac{1}{2}(0.28)^2$$
$$= \tfrac{1}{2}(0.0784)$$
$$\mathbf{L = 0.0392}$$

The loss is small, confirming that a prediction of 0.72 is reasonably close to the target of 1. For comparison, a perfect prediction $\hat{y} = 1$ would give $L = 0$, while a completely wrong prediction $\hat{y} = 0$ would give $L = 0.5$ — the maximum possible value of this loss for binary targets.

(ii) Gradient of the loss with respect to ŷ

Differentiate $L = \frac{1}{2}(y - \hat{y})^2$ with respect to $\hat{y}$, remembering the chain rule and that $d(y - \hat{y})/d\hat{y} = -1$:

$$\partial L/\partial \hat{y} = \tfrac{1}{2} \cdot 2(y - \hat{y}) \cdot (-1)$$
$$= -(y - \hat{y})$$
$$= \hat{y} - y$$
$$= 0.72 - 1$$
$$\mathbf{\partial L/\partial \hat{y} = -0.28}$$

Interpretation of the sign. The gradient is negative, which means the loss *decreases* as $\hat{y}$ increases. Since gradient descent moves parameters in the direction opposite to the gradient, the update will push $\hat{y}$ **upwards** towards the target value of 1 — exactly the desired behaviour.

(iii) Gradient with respect to the pre-activation z

First evaluate the derivative of the sigmoid at the current operating point:

$$\sigma'(z) = \hat{y}(1 - \hat{y})$$
$$= 0.72 \times (1 - 0.72)$$
$$= 0.72 \times 0.28$$
$$= 0.2016$$

Now apply the chain rule:

$$\partial L/\partial z = (\partial L/\partial \hat{y}) \cdot \hat{y}(1 - \hat{y})$$
$$= (-0.28) \times (0.2016)$$
$$\mathbf{\partial L/\partial z = -0.056448 \quad (\approx -0.0564)}$$

Summary of results

Quantity	Formula	Substitution	Result
Loss L	$\frac{1}{2}(y - \hat{y})^2$	$\frac{1}{2}(1 - 0.72)^2$	**0.0392**
$\partial L/\partial \hat{y}$	$\hat{y} - y$	$0.72 - 1$	**−0.28**
$\sigma'(z) = \hat{y}(1 - \hat{y})$	$\hat{y}(1 - \hat{y})$	0.72×0.28	**0.2016**
$\partial L/\partial z$	$(\partial L/\partial \hat{y}) \cdot \hat{y}(1 - \hat{y})$	$(-0.28)(0.2016)$	**−0.056448**

Important observation — gradient attenuation

Notice that $|\partial L/\partial z| = 0.0564$ is considerably smaller than $|\partial L/\partial \hat{y}| = 0.28$: the sigmoid derivative has shrunk the gradient by a factor of about **5**. This happens because $\sigma'(z) \leq 0.25$ for every z, so passing a gradient through a sigmoid *always* attenuates it. If the same neuron had been more confident — say $\hat{y} = 0.98$ — then $\sigma' = 0.98 \times 0.02 = 0.0196$ and $\partial L/\partial z$ would be only −0.000392, a further 144-fold reduction. This is a concrete, single-neuron demonstration of the **vanishing gradient problem** discussed in Question 5, and it is the principal reason why binary cross-entropy is preferred over squared error for classification (see Question 14).

Question 9 | Numerical — Single Neuron with Four Inputs and Four Activations

A single artificial neuron receives four inputs with the weights and bias shown in the figure. (i) Determine the net input (pre-activation) z of the neuron. (ii) Evaluate the neuron output if f is the sigmoid function. (iii) Estimate the neuron output if f is tanh, if f is ReLU, and if f is Leaky ReLU with a negative slope of 0.01. (iv) Explain which of these activations would stop the gradient from flowing back through this neuron, and why.

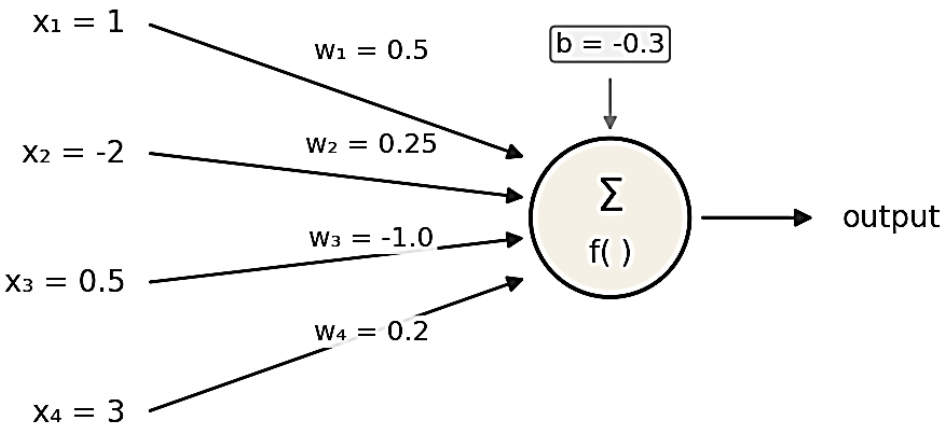


Figure 9.1 — Single neuron with four weighted inputs and a bias.

Given data (read from the figure)

Inputs : $x_1 = 1$, $x_2 = -2$, $x_3 = 0.5$, $x_4 = 3$

Weights : $w_1 = 0.5$, $w_2 = 0.25$, $w_3 = -1.0$, $w_4 = 0.2$

Bias : $b = -0.3$

(i) Net input (pre-activation) z

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$
$$= (0.5)(1) + (0.25)(-2) + (-1.0)(0.5) + (0.2)(3) + (-0.3)$$
$$= 0.5 + (-0.5) + (-0.5) + 0.6 - 0.3$$
$$= 1.1 - 1.3$$
$$\mathbf{z = -0.2}$$

The positive contributions (0.5 from x_1 and 0.6 from x_4) total +1.1, while the negative contributions (−0.5 from x_2, −0.5 from x_3 and −0.3 from the bias) total −1.3. The net input is therefore slightly negative, which turns out to be decisive for parts (iii) and (iv).

(ii) Output with sigmoid activation

$$\sigma(z) = 1 / (1 + e^{(-z)})$$
$$\sigma(-0.2) = 1 / (1 + e^{(0.2)})$$
$$= 1 / (1 + 1.2214)$$
$$= 1 / 2.2214$$
$$\mathbf{Output = 0.4502}$$

The value is just below 0.5, consistent with a slightly negative net input. Sigmoid never outputs exactly 0, so the neuron remains weakly active.

(iii) Outputs with tanh, ReLU and Leaky ReLU

Tanh:

```
tanh(z) = (e^z − e^(−z)) / (e^z + e^(−z))
tanh(−0.2) = (0.8187 − 1.2214) / (0.8187 + 1.2214)
           = (−0.4027) / (2.0401)
tanh output = −0.1974
```

ReLU:

```
ReLU(z) = max(0, z) = max(0, −0.2)
ReLU output = 0
```

Leaky ReLU (α = 0.01):

```
LeakyReLU(z) = z      if z > 0
             = α·z    if z ≤ 0
             = (0.01)(−0.2)
Leaky ReLU output = −0.002
```

Comparison of all four activations at z = −0.2

Activation	Output f(z)	Derivative f′(z)	Gradient flows?
Sigmoid	0.4502	σ(1−σ) = 0.4502 × 0.5498 = **0.2475**	Yes — but attenuated by ~4×
Tanh	−0.1974	1 − tanh² = 1 − 0.0390 = **0.9610**	Yes — almost undiminished
ReLU	0	0 (since z < 0)	**No — gradient is completely blocked**
Leaky ReLU	−0.002	α = **0.01**	Yes — but heavily attenuated (100×)

(iv) Which activation stops the gradient, and why

ReLU stops the gradient completely. The reason lies in its derivative:

```
d/dz ReLU(z) = 1  if z > 0
             = 0  if z < 0
```

Here z = −0.2 < 0, so ReLU′(z) = 0. During backpropagation the error term for this neuron is δ = (incoming gradient) × f′(z). Multiplying by zero annihilates the gradient, so:

- $\partial L/\partial w_1 = \partial L/\partial w_2 = \partial L/\partial w_3 = \partial L/\partial w_4 = 0$ and $\partial L/\partial b = 0$ — none of this neuron's parameters receive any update.
- No gradient is passed back to any neuron feeding into it, so an entire upstream sub-path is silenced for this input.
- If z remains negative for **every** training example, the neuron can never change its weights and is permanently disabled. This is the **dying ReLU problem**.

Why the others do not stop the gradient. Tanh has derivative 0.9610 at this point — close to its maximum of 1 — so it passes the gradient back almost intact and is the healthiest of the four here. Sigmoid has derivative 0.2475, near its maximum of 0.25, so it passes the gradient but shrinks it roughly fourfold, which compounds exponentially with depth. Leaky ReLU passes a gradient of α = 0.01: small, but crucially **non-zero**, so the neuron retains a route back to life. This is precisely the design motivation for Leaky ReLU, PReLU and ELU — they preserve a small negative-side slope so that no unit can ever become permanently dead.

Question 10 | Numerical — Parameter Counting in a 2–3–1 Network

> *Consider a two-layer neural network with 2 input neurons, 3 hidden neurons, and 1 output neuron. All connections are fully connected. (a) Draw the architecture. (b) Evaluate total number of weights. (c) Evaluate total trainable parameters when each hidden and output neuron has one bias. (d) Explain how the parameter count changes if hidden neurons increase from 3 to 5.*

(a) Architecture

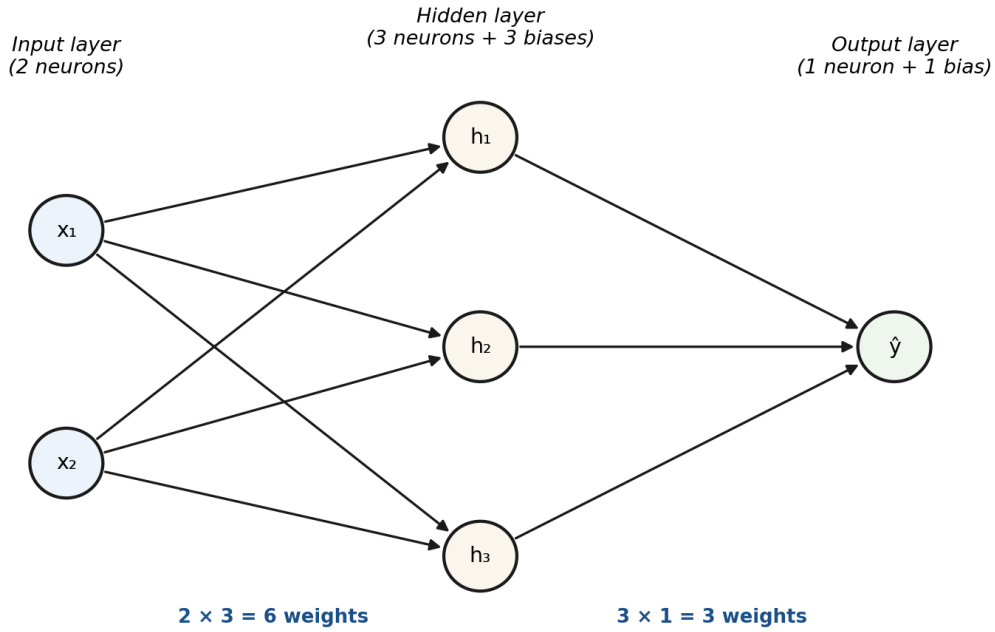


Figure 10.1 — Fully connected 2–3–1 architecture. Every input connects to every hidden neuron, and every hidden neuron connects to the output neuron.

Note that the input layer holds no biases and performs no computation, so it contributes no trainable parameters. The network is described as *two-layer* because only the hidden and output layers have weights — a common convention in which the input layer is not counted.

(b) Total number of weights

In a fully connected layer, the number of weights equals (number of neurons in the previous layer) × (number of neurons in the current layer).

```
Input → Hidden  : 2 × 3 = 6 weights
Hidden → Output : 3 × 1 = 3 weights
                  ──────────────────
```

Total weights = 6 + 3 = 9

(c) Total trainable parameters

Each hidden neuron and the output neuron carries one bias:

```
Hidden layer biases : 3
Output layer biases : 1
Total biases        : 4

Total trainable parameters = weights + biases
                           = 9 + 4
```

Total trainable parameters = 13

Layer-wise breakdown

Layer	Weights	Biases	Parameters
Input → Hidden	2 × 3 = 6	3	9
Hidden → Output	3 × 1 = 3	1	4
Total	**9**	**4**	**13**

(d) Effect of increasing hidden neurons from 3 to 5

Recompute with n = 5 hidden neurons:

```
Input → Hidden weights  : 2 × 5 = 10
Hidden → Output weights : 5 × 1 = 5
Total weights           : 15
Biases                  : 5 + 1 = 6
```

Total trainable parameters = 15 + 6 = 21

Hidden neurons (n)	Weights	Biases	Total parameters	Increase
3	9	4	13	—
5	15	6	21	+8
10	30	11	41	+28
100	300	101	401	+388

General formula and interpretation

For an input layer of size i, a hidden layer of size n and an output layer of size o:

Total parameters = (i·n + n) + (n·o + o) = n(i + o + 1) + o

With i = 2 and o = 1 this simplifies to **4n + 1**, which correctly yields 13 for n = 3 and 21 for n = 5.

- The relationship is **linear in n**: each additional hidden neuron adds i + o + 1 = 2 + 1 + 1 = 4 parameters (2 incoming weights, 1 outgoing weight, 1 bias).
- Going from 3 to 5 neurons adds 2 × 4 = 8 parameters, a **61.5% increase** in model size for only two extra units.
- More parameters increase the network's **capacity** to fit complex functions, but also increase memory use, training time, and the **risk of overfitting** when training data is limited.
- A common rule of thumb is that the number of training samples should comfortably exceed the number of parameters; otherwise regularisation (dropout, L2, early stopping) becomes essential.
- Note that widening a layer grows parameters linearly, whereas adding a *layer* between two wide layers grows them multiplicatively — which is why depth is generally a more parameter-efficient way to gain expressive power.

Question 11 | Numerical — Complete Forward Pass, Backpropagation and Weight Update

> *In the network below both neurons use the sigmoid activation. The loss is $L = \frac{1}{2}(y - \hat{y})^2$ and the learning rate is $\eta = 0.1$. (i) Perform the forward pass: calculate z_1, h, z_2 and $\hat{y}$, and hence the loss L. (ii) Calculate $\partial L/\partial \hat{y}$ and the output-layer error term $\delta_o = (\partial L/\partial \hat{y})\cdot\hat{y}(1 - \hat{y})$. (iii) Calculate $\partial L/\partial w_2$ and $\partial L/\partial b_2$. (iv) Calculate the hidden-layer error term δ_h and hence $\partial L/\partial w_1$ and $\partial L/\partial b_1$. (v) Update w_1 and w_2 using gradient descent, and comment on why the update to w_1 is so much smaller than the update to w_2.*

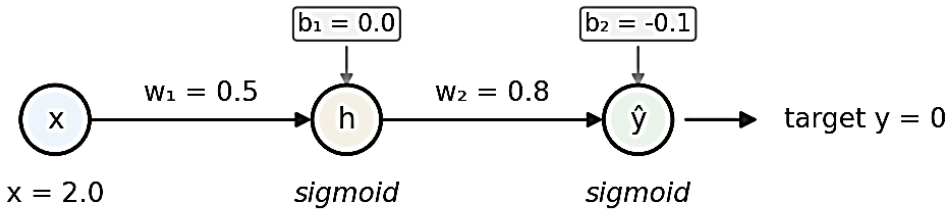


Figure 11.1 — Single-path network: $x \to h \to \hat{y}$, both neurons sigmoid.

Given data (read from the figure)

```
x  = 2.0        target y = 0        learning rate η = 0.1
w₁ = 0.5,  b₁ = 0.0     (input → hidden,  sigmoid)
w₂ = 0.8,  b₂ = −0.1    (hidden → output, sigmoid)
Loss: L = ½(y − ŷ)²
```

(i) Forward pass

Hidden neuron:

$$z_1 = w_1 \cdot x + b_1 = (0.5)(2.0) + 0.0 = 1.0$$

$$h = \sigma(1.0) = 1 / (1 + e^{-1.0}) = 1 / (1 + 0.3679) = 1 / 1.3679$$

$$\mathbf{h = 0.7311}$$

Output neuron:

$$z_2 = w_2 \cdot h + b_2 = (0.8)(0.7311) + (-0.1)$$

$$= 0.5849 - 0.1 = 0.4849$$

$$\hat{y} = \sigma(0.4849) = 1 / (1 + e^{-0.4849}) = 1 / (1 + 0.6158)$$

$$\mathbf{\hat{y} = 0.6189}$$

Loss:

$$L = \tfrac{1}{2}(y - \hat{y})^2 = \tfrac{1}{2}(0 - 0.6189)^2 = \tfrac{1}{2}(0.3830)$$

$$\mathbf{L = 0.1915}$$

The prediction 0.6189 is badly wrong — the target is 0 — so a substantial loss and a large corrective gradient are expected.

(ii) Output-layer gradients

$$\partial L/\partial \hat{y} = \hat{y} - y = 0.6189 - 0 = 0.6189$$

$$\sigma'(z_2) = \hat{y}(1 - \hat{y}) = 0.6189 \times 0.3811 = 0.2359$$

$\delta_o = (\partial L/\partial \hat{y}) \cdot \hat{y}(1 - \hat{y}) = 0.6189 \times 0.2359$

$\delta_o = 0.1460$

The gradient is positive, so gradient descent will *decrease* $\hat{y}$ — correct, since the target is 0.

(iii) Gradients of the output-layer parameters

Since $z_2 = w_2 h + b_2$, we have $\partial z_2/\partial w_2 = h$ and $\partial z_2/\partial b_2 = 1$:

$\partial L/\partial w_2 = \delta_o \cdot h = 0.1460 \times 0.7311$

$\partial L/\partial w_2 = 0.1067$

$\partial L/\partial b_2 = \delta_o \times 1$

$\partial L/\partial b_2 = 0.1460$

(iv) Hidden-layer error term and gradients

The hidden neuron affects the loss only through the output neuron, so the error must be back-projected through w_2 and then gated by the hidden neuron's own sigmoid derivative:

$\sigma'(z_1) = h(1 - h) = 0.7311 \times 0.2689 = 0.1966$

$\delta_h = \delta_o \cdot w_2 \cdot h(1 - h)$
$= 0.1460 \times 0.8 \times 0.1966$
$= 0.1168 \times 0.1966$

$\delta_h = 0.02296$

$\partial L/\partial w_1 = \delta_h \cdot x = 0.02296 \times 2.0$

$\partial L/\partial w_1 = 0.04592$

$\partial L/\partial b_1 = \delta_h \times 1$

$\partial L/\partial b_1 = 0.02296$

(v) Gradient-descent updates

$w_2(\text{new}) = w_2 - \eta \cdot \partial L/\partial w_2 = 0.8 - (0.1)(0.1067)$
$= 0.8 - 0.01067$

$w_2(\text{new}) = 0.7893 \quad \rightarrow \quad |\Delta w_2| = 0.01067$

$w_1(\text{new}) = w_1 - \eta \cdot \partial L/\partial w_1 = 0.5 - (0.1)(0.04592)$
$= 0.5 - 0.004592$

$w_1(\text{new}) = 0.4954 \quad \rightarrow \quad |\Delta w_1| = 0.00459$

For completeness, the biases update to $b_2(\text{new}) = -0.1 - (0.1)(0.1460) =$ **-0.1146** and $b_1(\text{new}) = 0.0 - (0.1)(0.02296) =$ **-0.0023**.

Complete summary

Quantity	Value	Quantity	Value
z_1	1.0000	δ_o	0.1460
h	0.7311	$\partial L/\partial w_2$	0.1067
z_2	0.4849	$\partial L/\partial b_2$	0.1460
$\hat{y}$	0.6189	δ_h	0.02296
L	0.1915	$\partial L/\partial w_1$	0.04592
$\partial L/\partial \hat{y}$	0.6189	$\partial L/\partial b_1$	0.02296

Quantity	Value	Quantity	Value
w_2 : 0.8 → 0.7893	Δ = −0.01067	**w_1 : 0.5 → 0.4954**	Δ = −0.00459

Why the update to w_1 is so much smaller than the update to w_2

The update to w_2 is $|\Delta w_2| = 0.01067$ while the update to w_1 is $|\Delta w_1| = 0.00459$ — roughly **2.3 times smaller** — even though x = 2.0 actually *amplifies* the w_1 gradient by a factor of 2. The reason is that the gradient reaching the hidden layer has been attenuated twice on its way back:

- **Attenuation by the weight $w_2 = 0.8$.** The error is back-projected through the connection, multiplying it by 0.8, an immediate 20% reduction. Any $|w| < 1$ shrinks the signal at this step.
- **Attenuation by the sigmoid derivative $h(1 - h) = 0.1966$.** This is by far the dominant effect: it reduces the gradient by a factor of about 5. The sigmoid derivative can never exceed 0.25, so *every* sigmoid layer traversed shrinks the gradient by at least 4×, and typically much more.
- **Net effect:** $\delta_h = \delta_o \times 0.8 \times 0.1966 = \delta_o \times 0.157$, i.e. the hidden layer's error term is only about 15.7% of the output layer's. The subsequent multiplication by x = 2.0 recovers some of this, leaving the final ratio at 1 : 2.3.

Wider significance. This is the **vanishing gradient problem** observed directly, in a network only two layers deep. If the network had ten sigmoid layers instead of two, the compounding factor would be roughly $0.157^{10} \approx 9 \times 10^{-9}$, so the first layer's weights would be effectively frozen. The layers nearest the output learn quickly while the layers nearest the input barely move — which is why deep networks use ReLU (derivative exactly 1 in the active region), careful initialisation, batch normalisation, and residual connections instead of stacks of sigmoids.

Question 12 | Numerical — Forward Pass with ReLU Hidden Layer and Sigmoid Output

The network below is fully specified. All hidden neurons use ReLU and the output neuron uses the sigmoid function. (i) Compute the pre-activation values z_h1 and z_h2, and hence the hidden-layer outputs h_1 and h_2. (ii) Compute the pre-activation value at the output neuron and the final network output $\hat{y}$. (iii) If the target output is y = 1 and the loss is $L = \frac{1}{2}(y - \hat{y})^2$, calculate the loss. (iv) State what the value of h_1 would become if the bias b = 0.1 were changed to −0.5, and explain what this reveals about ReLU.

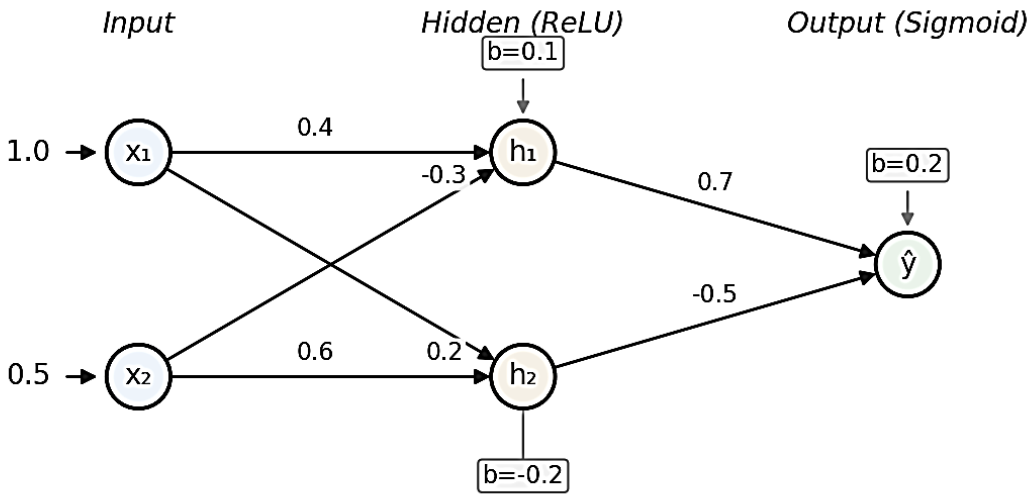


Figure 12.1 — 2–2–1 network with ReLU hidden units and a sigmoid output unit.

Given data (read from the figure)

Inputs : $x_1 = 1.0$, $x_2 = 0.5$

$x_1 \to h_1 = 0.4$	$x_2 \to h_1 = -0.3$	b_h1 = 0.1
$x_1 \to h_2 = 0.2$	$x_2 \to h_2 = 0.6$	b_h2 = −0.2
$h_1 \to \hat{y} = 0.7$	$h_2 \to \hat{y} = -0.5$	b_o = 0.2
Hidden activation: ReLU		Output activation: Sigmoid

(i) Hidden-layer pre-activations and outputs

Hidden neuron h_1:

$$z_{h1} = (x_1 \times 0.4) + (x_2 \times -0.3) + b_{h1}$$
$$= (1.0)(0.4) + (0.5)(-0.3) + 0.1$$
$$= 0.4 - 0.15 + 0.1$$
$$\mathbf{z_{h1} = 0.35 \quad \to \quad h_1 = ReLU(0.35) = 0.35}$$

Hidden neuron h_2:

$$z_{h2} = (x_1 \times 0.2) + (x_2 \times 0.6) + b_{h2}$$
$$= (1.0)(0.2) + (0.5)(0.6) + (-0.2)$$
$$= 0.2 + 0.3 - 0.2$$
$$\mathbf{z_{h2} = 0.30 \quad \to \quad h_2 = ReLU(0.30) = 0.30}$$

Both pre-activations are positive, so both ReLU units pass their input through unchanged and both are active.

(ii) Output pre-activation and final output

$$z_o = (h_1 \times 0.7) + (h_2 \times -0.5) + b_o$$
$$= (0.35)(0.7) + (0.30)(-0.5) + 0.2$$

```
    = 0.245 − 0.150 + 0.2
z_o = 0.295
ŷ = σ(0.295) = 1 / (1 + e^(−0.295))
  = 1 / (1 + 0.7445)
  = 1 / 1.7445
ŷ = 0.5732
```

(iii) Loss

```
L = ½(y − ŷ)²
  = ½(1 − 0.5732)²
  = ½(0.4268)²
  = ½(0.18216)
L = 0.0911
```

The prediction 0.5732 is only marginally on the correct side of the 0.5 decision threshold, so the sample would be classified as class 1 (correctly), but with very low confidence — hence the substantial loss.

Summary of the forward pass

Neuron	Pre-activation z	Activation	Output
h_1	0.35	ReLU	**0.35**
h_2	0.30	ReLU	**0.30**
$\hat{y}$	0.295	Sigmoid	**0.5732**
Loss (y = 1)	—	$\frac{1}{2}(y - \hat{y})^2$	**0.0911**

(iv) Effect of changing the bias from 0.1 to −0.5

Recompute the pre-activation of h_1 with the new bias:

```
z_h1 = (1.0)(0.4) + (0.5)(−0.3) + (−0.5)
     = 0.4 − 0.15 − 0.5
     = −0.25
h₁ = ReLU(−0.25) = max(0, −0.25)
h₁ = 0
```

What this reveals about ReLU

- **ReLU is a hard gate, not a soft squashing function.** The moment the pre-activation crosses zero, the output snaps to exactly 0. There is no gradual decay as there would be with sigmoid or tanh — a sigmoid would still have returned σ(−0.25) = 0.4378, keeping the neuron partially active.
- **The bias directly controls the firing threshold.** The neuron fires only when $0.4x_1 - 0.3x_2 + b > 0$. Lowering b from 0.1 to −0.5 raises the threshold that the weighted input must clear, silencing the neuron for this particular input. Biases are therefore not cosmetic — they decide *which* inputs a feature detector responds to.
- **ReLU produces sparse representations.** With $h_1 = 0$, the hidden layer's output vector becomes [0, 0.30]: half the representation is switched off. In a large layer typically 50% or more of units are inactive for any given input, which is computationally efficient and acts as an implicit regulariser.

- **The gradient is completely blocked through a zeroed unit.** Since ReLU′(z) = 0 for z < 0, backpropagation gives ∂L/∂w = 0 and ∂L/∂b = 0 for every parameter of h_1, so this neuron receives *no* update from this training example — a direct illustration of the mechanism behind the **dying ReLU** problem.
- **Downstream effect.** The output pre-activation would become z_o = (0)(0.7) + (0.30)(−0.5) + 0.2 = 0.05, giving ŷ = σ(0.05) = 0.5125 — the prediction moves noticeably closer to the undecided value of 0.5, showing how switching off a single hidden feature degrades the network's confidence.
- **Practical implication.** If a unit's pre-activation is negative for *every* training sample, it is permanently dead and its parameters are wasted capacity. Leaky ReLU (f(z) = 0.01z for z < 0), PReLU or ELU are used precisely to keep a small non-zero gradient so that such a unit can recover.

ADDITIONAL QUESTIONS (13 – 17)

Question 13 | The Perceptron, its Limitations, and the XOR Problem

> *Explain the perceptron model and its learning rule. Discuss the limitation of a single-layer perceptron with reference to the XOR problem, and show how a multilayer network overcomes it.*

1. The perceptron model

The **perceptron**, introduced by Frank Rosenblatt in 1958, is the earliest trainable neural model and the direct ancestor of the modern artificial neuron. It is a binary classifier that computes a weighted sum of its inputs and passes the result through a hard threshold (step) function:

$$z = \Sigma_i\, w_i x_i + b$$
$$\hat{y} = \text{step}(z) = 1 \quad \text{if } z \geq 0$$
$$= 0 \quad \text{if } z < 0$$

Geometrically, the equation $w^T x + b = 0$ defines a **hyperplane** that splits the input space into two half-spaces. Points on one side are labelled 1 and points on the other side 0. In two dimensions this hyperplane is simply a straight line.

2. The perceptron learning rule

The perceptron is trained by a remarkably simple error-correction rule. For each training sample (x, y), predict $\hat{y}$ and update only when the prediction is wrong:

$$\text{error } e = y - \hat{y}$$
$$w_i \leftarrow w_i + \eta \cdot e \cdot x_i$$
$$b \leftarrow b + \eta \cdot e$$

If the prediction is correct, e = 0 and nothing changes. If the true label is 1 but the perceptron predicted 0, then e = +1 and the weights move *towards* the input, making the neuron more likely to fire on similar patterns next time; the reverse happens when e = −1.

The **Perceptron Convergence Theorem** (Novikoff, 1962) guarantees that if the training data is linearly separable, this rule will find a separating hyperplane in a finite number of steps. Crucially, if the data is *not* linearly separable, the algorithm never converges — it oscillates indefinitely.

3. The limitation: linear separability

A single-layer perceptron can only represent functions whose positive and negative examples can be separated by a single straight line (or hyperplane). This restriction was made famous by Minsky and Papert's 1969 book *Perceptrons*, and it triggered the first 'AI winter'. The AND, OR and NOT functions are all linearly separable and therefore learnable; **XOR is not**.

4. Why XOR cannot be solved by a single perceptron

The XOR (exclusive-OR) truth table is:

x_1	x_2	XOR output	Position in the plane
0	0	0	bottom-left
0	1	1	top-left

x_1	x_2	XOR output	Position in the plane
1	0	1	bottom-right
1	1	0	top-right

The two positive points (0,1) and (1,0) lie on one diagonal of the unit square, and the two negative points (0,0) and (1,1) lie on the other. Any straight line placed in this square must leave at least one point misclassified — the classes are *diagonally interleaved*.

Formal proof by contradiction. Suppose weights w_1, w_2 and bias b exist such that step($w_1x_1 + w_2x_2 + b$) reproduces XOR. Then:

$(0,0) \to 0$ requires $b < 0$ …(1)
$(0,1) \to 1$ requires $w_2 + b \geq 0$, so $w_2 \geq -b$ …(2)
$(1,0) \to 1$ requires $w_1 + b \geq 0$, so $w_1 \geq -b$ …(3)
$(1,1) \to 0$ requires $w_1 + w_2 + b < 0$ …(4)

Adding (2) and (3): $w_1 + w_2 \geq -2b$. Substituting into (4): $-2b + b < 0$, i.e. $-b < 0$, i.e. $b > 0$. This contradicts (1), which requires $b < 0$. **No such weights exist**, so a single perceptron can never compute XOR. The best achievable accuracy is 75% (three of the four points).

5. How a multilayer network solves XOR

The key insight is that XOR *can* be decomposed into linearly separable sub-problems. Note the identity **XOR = OR AND (NOT AND)** — a point is XOR-positive if at least one input is 1 but not both. A hidden layer can compute the intermediate features OR and AND (each individually linearly separable), and the output layer then combines them linearly.

An explicit ReLU solution with two hidden neurons is:

$h_1 = \mathrm{ReLU}(x_1 + x_2)$ (weights 1, 1; bias 0)
$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$ (weights 1, 1; bias −1)
$\hat{y} = h_1 - 2 \cdot h_2$ (weights 1, −2; bias 0)

Verification on all four inputs:

(x_1, x_2)	$x_1 + x_2$	$h_1 = \mathrm{ReLU}(\cdot)$	$h_2 = \mathrm{ReLU}(\cdot - 1)$	$\hat{y} = h_1 - 2h_2$	XOR target
(0, 0)	0	0	0	0 − 0 = **0**	0 ✓
(0, 1)	1	1	0	1 − 0 = **1**	1 ✓
(1, 0)	1	1	0	1 − 0 = **1**	1 ✓
(1, 1)	2	2	1	2 − 2 = **0**	0 ✓

All four cases are correct. Geometrically, the hidden layer performs a **nonlinear change of coordinates**: it maps the four input points into a new feature space (h_1, h_2) in which (0,1) and (1,0) collapse onto the *same* point (1, 0), so the two classes become linearly separable and a single output neuron suffices.

6. Comparison of perceptron and MLP

Aspect	Single-layer Perceptron	Multilayer Perceptron
Activation	Hard step function (non-differentiable)	Smooth or piecewise-linear (sigmoid, tanh, ReLU)
Training algorithm	Perceptron learning rule (error correction)	Backpropagation + gradient descent
Decision boundary	A single hyperplane	Arbitrary nonlinear, piecewise-linear surfaces
Representable functions	Only linearly separable functions (AND, OR, NOT)	Any continuous function (Universal Approximation Theorem)
XOR	Impossible — maximum 75% accuracy	Solved exactly with 2 hidden neurons
Output	Binary (0 or 1) only	Continuous probabilities or real values

Historical note. The XOR limitation stalled neural-network research for nearly two decades. It was resolved not by abandoning the perceptron but by (a) stacking layers and (b) replacing the non-differentiable step function with a differentiable activation, which made gradient-based training via backpropagation possible — popularised by Rumelhart, Hinton and Williams in 1986.

Question 14 | Loss Functions and Why Cross-Entropy Beats MSE for Classification

> *Explain the role of a loss function in training a neural network. Compare Mean Squared Error with Binary and Categorical Cross-Entropy, and derive mathematically why cross-entropy is preferred over squared error when the output neuron uses a sigmoid activation.*

1. Role of the loss function

The **loss function** (also called the cost or objective function) quantifies the discrepancy between the network's prediction $\hat{y}$ and the true target y as a single scalar. It serves three purposes: it defines *what* 'good' means for the model, it provides the scalar whose gradient backpropagation computes, and it shapes the geometry of the optimisation landscape and therefore how easily the network trains. Training is precisely the problem of minimising the average loss over the training set:

$$J(W, b) = (1/N)\ \Sigma_{n=1}^{N}\ L(y^{(n)}, \hat{y}^{(n)})$$

A well-chosen loss must be differentiable (so gradients exist), must be minimised exactly when the prediction is correct, and should produce gradients of useful magnitude — neither vanishing nor exploding — across the whole range of possible predictions.

2. The principal loss functions

Loss function	Formula	Used for
Mean Squared Error (MSE)	$L = (1/N)\Sigma(y - \hat{y})^2$	Regression with roughly Gaussian noise. Heavily penalises large errors, so it is sensitive to outliers.
Mean Absolute Error (MAE)	$L = (1/N)\Sigma\lvert y - \hat{y}\rvert$	Regression where robustness to outliers matters. Gradient is constant, so convergence near the minimum can be slow.
Huber loss	quadratic for small errors, linear for large	Regression; a compromise between MSE and MAE.
Binary Cross-Entropy (BCE)	$L = -[y\cdot\log \hat{y} + (1-y)\cdot\log(1-\hat{y})]$	Binary classification, paired with a sigmoid output.
Categorical Cross-Entropy	$L = -\Sigma_k\ y_k\cdot\log \hat{y}_k$	Multi-class, single-label classification, paired with a softmax output.
Hinge loss	$L = \max(0, 1 - y\cdot\hat{y})$	Maximum-margin classifiers (SVM-style), with $y \in \{-1, +1\}$.

3. Intuition for cross-entropy

Cross-entropy comes from information theory: it measures how many extra bits are needed to encode the true distribution using the predicted distribution. For a binary target, only one of the two terms in the BCE formula survives:

$$\text{If } y = 1 :\ \ L = -\log(\hat{y})$$
$$\text{If } y = 0 :\ \ L = -\log(1 - \hat{y})$$

The penalty is therefore *unbounded*: predicting $\hat{y} = 0.01$ when $y = 1$ gives $L = -\log(0.01) = 4.61$, whereas MSE would give only $\tfrac{1}{2}(0.99)^2 = 0.49$. Cross-entropy punishes confident wrong predictions extremely harshly, which is exactly the behaviour we want from a classifier. It is also the **maximum-likelihood** loss for a Bernoulli (or categorical) output model, giving it a principled statistical justification.

4. Derivation — why cross-entropy is preferred with a sigmoid output

Case A: MSE with a sigmoid output. Let $\hat{y} = \sigma(z)$ and $L = \frac{1}{2}(y - \hat{y})^2$.

$$\partial L/\partial \hat{y} = \hat{y} - y$$
$$\partial \hat{y}/\partial z = \sigma'(z) = \hat{y}(1 - \hat{y})$$
$$\partial L/\partial z = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

Case B: Binary cross-entropy with a sigmoid output. Let $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.

$$\partial L/\partial \hat{y} = -y/\hat{y} + (1 - y)/(1 - \hat{y})$$
$$= [-y(1 - \hat{y}) + (1 - y)\hat{y}] / [\hat{y}(1 - \hat{y})]$$
$$= (\hat{y} - y) / [\hat{y}(1 - \hat{y})]$$

Now multiply by $\partial \hat{y}/\partial z = \hat{y}(1 - \hat{y})$. The denominator **cancels exactly**:

$$\partial L/\partial z = (\hat{y} - y)$$ **— the saturating factor has vanished**

This cancellation is the mathematical heart of the matter. With cross-entropy, the gradient at the output neuron is simply the prediction error, independent of the sigmoid derivative.

5. Numerical demonstration of the difference

Consider a badly wrong, highly confident prediction: the true label is $y = 0$ but the network outputs $\hat{y} = 0.98$.

$$\text{MSE} : \partial L/\partial z = (0.98 - 0)(0.98)(0.02) = 0.0192$$
$$\text{Cross-entropy} : \partial L/\partial z = (0.98 - 0) = 0.9800$$

Cross-entropy delivers a gradient ≈ 51× larger

With MSE, the neuron is *maximally wrong* yet receives an almost negligible corrective signal, because $\sigma'(z) \approx 0.0196$ in the saturated region. Learning stalls precisely when it is most needed. Cross-entropy produces a large gradient exactly when the error is large, so recovery is fast.

6. Additional reasons for the preference

- **Convexity.** With respect to the output-layer pre-activations, cross-entropy paired with sigmoid/softmax is convex, whereas MSE paired with sigmoid is non-convex and introduces spurious local minima.
- **Probabilistic interpretation.** Minimising cross-entropy is equivalent to maximising the likelihood of the data under a Bernoulli (binary) or categorical (multi-class) model, so the outputs are properly calibrated probabilities.
- **Correct error scaling.** MSE treats a 0.1 error the same whether it occurs near the decision boundary or at the extremes; cross-entropy weights errors by how confident and how wrong the model was.
- **The same cancellation holds for softmax.** For softmax outputs with categorical cross-entropy, $\partial L/\partial z_k = \hat{y}_k - y_k$ (demonstrated numerically in Question 16), which is why this pairing is universal in multi-class classification.

7. Practical selection guide

Task	Output activation	Loss function
Binary classification	Sigmoid (1 neuron)	Binary cross-entropy
Multi-class, single label	Softmax (k neurons)	Categorical cross-entropy
Multi-label classification	Sigmoid (k independent neurons)	Binary cross-entropy applied per label
Regression	Linear / identity	MSE (or Huber if outliers are present)

Implementation note. In practice, libraries fuse the activation and the loss (e.g. `BCEWithLogitsLoss`, `softmax_cross_entropy_with_logits`) because computing log σ(z) directly, rather than log of a separately computed σ(z), is far more numerically stable and avoids overflow when |z| is large.

Question 15 | Gradient Descent Variants, Learning Rate, and Optimisers

> *Explain gradient descent as an optimisation algorithm. Compare batch, stochastic and mini-batch gradient descent, discuss the effect of the learning rate, and describe how momentum, RMSProp and Adam improve on plain gradient descent.*

1. Gradient descent — the basic idea

Gradient descent is a first-order iterative optimisation algorithm that minimises the loss by repeatedly stepping in the direction of steepest descent. Since the gradient ∇J points in the direction of steepest *increase*, we move in the opposite direction:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta J(\theta)$$

where θ denotes all trainable parameters (weights and biases) and η is the **learning rate**. Backpropagation supplies ∇J; gradient descent decides what to do with it. The process repeats until the loss converges, the gradient becomes negligibly small, or a fixed budget of epochs is exhausted.

2. The three variants

Variant	Samples per update	Advantages	Disadvantages
Batch (full-batch) GD	The entire training set (N)	Gradient estimate is exact and low-variance; smooth, stable, monotonic convergence; deterministic and easy to analyse	Extremely slow for large N; the whole dataset must fit in memory; only one update per epoch; tends to get trapped in sharp local minima and saddle points
Stochastic GD (SGD)	1 sample	Very fast per update; enables online learning from streaming data; the noise in the gradient helps escape shallow local minima and saddle points; minimal memory	Very noisy trajectory; the loss oscillates and may never settle exactly at the minimum; cannot exploit vectorised hardware efficiently
Mini-batch GD	A batch of m samples (typically 32–256)	Best of both: far less noisy than SGD, far faster than batch GD; makes full use of GPU parallelism; retains just enough noise for good generalisation	Introduces the batch size as an extra hyperparameter; too small a batch is noisy, too large loses the regularising benefit of noise

Mini-batch gradient descent is the standard in practice, and confusingly it is what most libraries simply call 'SGD'. Batch sizes are usually powers of two (32, 64, 128, 256) to align with GPU memory architecture.

3. Epochs, iterations and batches — a worked example

For a dataset of 10,000 samples trained with a batch size of 100 for 50 epochs:

```
Iterations per epoch = 10,000 / 100 = 100
Total parameter updates = 100 × 50 = 5,000
```

Under full-batch gradient descent the same 50 epochs would produce only **50** updates; under pure SGD they would produce 500,000. This illustrates why mini-batching strikes the practical balance.

4. Effect of the learning rate

Learning rate	Behaviour
Too small (e.g. 10^{-6})	Convergence is extremely slow; training may appear stalled; the optimiser is easily trapped in shallow local minima or plateaus because it lacks the energy to escape.
Well chosen (e.g. 10^{-3})	Loss decreases steadily and smoothly, reaching a good minimum within a reasonable number of epochs.
Too large (e.g. 1.0)	Steps overshoot the minimum; the loss oscillates, increases, or diverges to NaN. Weights can be driven into saturated or dead regions from which recovery is impossible.

Because no single value is ideal throughout training, **learning-rate schedules** are used: step decay (multiply η by 0.1 every k epochs), exponential decay, cosine annealing, and a short *warm-up* phase at the start of training. A practical diagnostic is the *learning-rate range test*: train for a few hundred iterations while increasing η exponentially and plot the loss; the best η is roughly an order of magnitude below the point at which the loss begins to diverge.

5. Advanced optimisers

(a) Momentum. Plain gradient descent zig-zags across narrow ravines. Momentum accumulates an exponentially weighted moving average of past gradients, damping oscillations across the ravine and accelerating progress along it:

$$v \leftarrow \beta \cdot v + (1 - \beta) \cdot \nabla J \qquad (\beta \approx 0.9)$$
$$\theta \leftarrow \theta - \eta \cdot v$$

(b) RMSProp. Different parameters may need very different step sizes. RMSProp divides each parameter's update by the root-mean-square of its recent gradients, so parameters with consistently large gradients take smaller steps and rarely-updated parameters take larger ones:

$$s \leftarrow \beta \cdot s + (1 - \beta) \cdot (\nabla J)^2$$
$$\theta \leftarrow \theta - \eta \cdot \nabla J \,/\, (\sqrt{s} + \varepsilon)$$

(c) Adam (Adaptive Moment Estimation). Adam combines both ideas, tracking the first moment (mean, like momentum) and the second moment (uncentred variance, like RMSProp), with a bias correction for the initialisation at zero:

$$m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot \nabla J \qquad (\beta_1 = 0.9)$$
$$s \leftarrow \beta_2 \cdot s + (1 - \beta_2) \cdot (\nabla J)^2 \qquad (\beta_2 = 0.999)$$
$$\hat{m} = m \,/\, (1 - \beta_1^{\,t}), \quad \hat{s} = s \,/\, (1 - \beta_2^{\,t})$$
$$\theta \leftarrow \theta - \eta \cdot \hat{m} \,/\, (\sqrt{\hat{s}} + \varepsilon) \qquad (\varepsilon = 10^{-8})$$

Adam is the most widely used optimiser for deep networks because it converges quickly, is relatively insensitive to the initial learning rate, handles sparse gradients well, and requires little tuning. Its usual default is η = 0.001. Well-tuned SGD with momentum sometimes generalises slightly better on large vision benchmarks, and **AdamW** — Adam with correctly decoupled weight decay — is now the standard choice for training Transformers.

6. Comparison summary

Optimiser	Adaptive step size?	Uses momentum?	Typical default η
SGD	No	No	0.01 – 0.1
SGD + Momentum	No	Yes	0.01 – 0.1 (β = 0.9)

Optimiser	Adaptive step size?	Uses momentum?	Typical default η
RMSProp	Yes	No	0.001
Adam / AdamW	Yes	Yes	0.001

Question 16 | Numerical — Softmax Output Layer and Categorical Cross-Entropy

> *A three-class classification network produces the output-layer logits z = [2.0, 1.0, 0.1]. The true class is class 1, so the one-hot target is y = [1, 0, 0]. (i) Compute the softmax probabilities. (ii) Compute the categorical cross-entropy loss. (iii) Compute the gradient of the loss with respect to each logit. (iv) Explain the numerical-stability trick used when implementing softmax, and why softmax is preferred over independent sigmoids for single-label multi-class problems.*

Given data

Logits : $z = [z_1, z_2, z_3] = [2.0, 1.0, 0.1]$

Target : $y = [1, 0, 0]$ (true class = class 1)

Softmax : $\hat{y}_k = e^{(z_k)} / \Sigma_j e^{(z_j)}$

Loss : $L = -\Sigma_k y_k \cdot \log(\hat{y}_k)$

(i) Softmax probabilities

Step 1 — exponentiate each logit:

$e^{(2.0)} = 7.3891$

$e^{(1.0)} = 2.7183$

$e^{(0.1)} = 1.1052$

Step 2 — sum the exponentials:

$\Sigma = 7.3891 + 2.7183 + 1.1052 = 11.2126$

Step 3 — normalise:

$\hat{y}_1 = 7.3891 / 11.2126 = 0.6590$

$\hat{y}_2 = 2.7183 / 11.2126 = 0.2424$

$\hat{y}_3 = 1.1052 / 11.2126 = 0.0986$

$\hat{y} = [0.6590, 0.2424, 0.0986]$ (sum = 1.0000 ✓)

The outputs form a valid probability distribution: every value lies in (0, 1) and they sum to exactly 1. The model assigns 65.9% probability to class 1, which is the correct class, so the prediction is correct but only moderately confident.

(ii) Categorical cross-entropy loss

Because the target is one-hot, all terms except the true class vanish:

$L = -[y_1 \cdot \log \hat{y}_1 + y_2 \cdot \log \hat{y}_2 + y_3 \cdot \log \hat{y}_3]$

$= -[1 \times \log(0.6590) + 0 \times \log(0.2424) + 0 \times \log(0.0986)]$

$= -\log(0.6590)$

$= -(-0.4170)$

$L = 0.4170$ nats

For reference: a perfect prediction ($\hat{y}_1 = 1.0$) would give L = 0, while a uniform guess ($\hat{y}_1 = 1/3$) would give L = −log(0.333) = 1.0986. The obtained loss of 0.4170 therefore represents a meaningful but incomplete improvement over random guessing.

(iii) Gradient with respect to the logits

For the softmax–cross-entropy pairing, the gradient simplifies to the elegant and exact result:

$\partial L / \partial z_k = \hat{y}_k - y_k$

This is the multi-class analogue of the cancellation derived in Question 14. Substituting:

$\partial L/\partial z_1 = 0.6590 - 1 = -0.3410$

$\partial L/\partial z_2 = 0.2424 - 0 = +0.2424$

$\partial L/\partial z_3 = 0.0986 - 0 = +0.0986$

$\nabla_z L = [-0.3410, +0.2424, +0.0986]$ (sum = 0 ✓)

Summary and interpretation

Class k	Logit z_k	$e^{(z_k)}$	Softmax $\hat{y}_k$	Target y_k	$\partial L/\partial z_k = \hat{y}_k - y_k$
1 (true)	2.0	7.3891	0.6590	1	**−0.3410**
2	1.0	2.7183	0.2424	0	**+0.2424**
3	0.1	1.1052	0.0986	0	**+0.0986**
Total	—	11.2126	**1.0000**	1	**0.0000**

The sign pattern is exactly what learning requires. The gradient for the true class is **negative**, so gradient descent will *increase* z_1 and push $\hat{y}_1$ towards 1. The gradients for the wrong classes are **positive**, so their logits are *decreased*. The gradients also sum to zero, reflecting the constraint that the probabilities must always sum to 1 — softmax redistributes probability mass rather than creating it. Note further that class 2, which was the more serious competitor (0.2424), receives a larger corrective push than class 3 (0.0986): the loss automatically focuses effort on the most confusable class.

(iv) Numerical stability and choice of softmax

The max-subtraction trick. Computing $e^{(z_k)}$ directly overflows for large logits — $e^{(1000)}$ is infinity in floating-point arithmetic — and underflows to zero for very negative ones. Softmax is invariant to adding any constant c to every logit, because the constant factors out of numerator and denominator:

$$\text{softmax}(z + c)_k = e^{(z_k+c)} / \Sigma_j\, e^{(z_j+c)} = e^c \cdot e^{(z_k)} / (e^c \cdot \Sigma_j\, e^{(z_j)}) = \text{softmax}(z)_k$$

Implementations therefore choose c = −max(z), so the largest exponent becomes $e^0 = 1$ and no overflow is possible. Verifying with our data (max = 2.0):

```
Shifted logits : [0.0, -1.0, -1.9]
Exponentials   : [1.0000, 0.3679, 0.1496]   Σ = 1.5175
Probabilities  : [0.6590, 0.2424, 0.0986]   — identical ✓
```

Why softmax rather than independent sigmoids? Softmax couples the outputs so that they compete for a fixed total probability of 1, which correctly encodes the assumption that the classes are **mutually exclusive** — exactly one label is right. Three independent sigmoids could output [0.9, 0.9, 0.9], which is meaningless for a single-label problem. Softmax also yields the clean gradient $\hat{y} - y$ and is the maximum-likelihood estimator for a categorical distribution. Independent sigmoids remain the correct choice for **multi-label** problems, where an input may legitimately belong to several classes at once (for example, an image tagged both 'beach' and 'sunset').

Question 17 | Weight Initialisation, Overfitting, and Regularisation

> *Explain why weight initialisation matters in deep neural networks and compare Xavier and He initialisation. Then distinguish overfitting from underfitting and describe the main regularisation techniques used to control them.*

PART A — WEIGHT INITIALISATION

1. Why initialisation matters

The starting point of optimisation determines both whether training converges and how fast. Because backpropagation multiplies signals layer by layer, the *scale* of the initial weights controls whether activations and gradients maintain a healthy magnitude as they propagate. A poor choice makes a deep network untrainable no matter how good the optimiser is.

2. What goes wrong with naive schemes

Initialisation	What happens	Why it fails
All zeros	Every neuron in a layer computes an identical output and receives an identical gradient, so they remain identical forever.	The **symmetry problem**: a layer of n neurons behaves like a single neuron, destroying the network's capacity. Biases *may* safely be set to zero, but weights must not be.
All equal to the same constant	Same as above.	Symmetry is again never broken.
Too small (e.g. $N(0, 0.01^2)$)	Activation variance shrinks layer by layer until signals are effectively zero.	The forward signal vanishes, and gradients vanish with it — the network stops learning.
Too large (e.g. $N(0, 1^2)$)	Activation variance grows layer by layer; sigmoid/tanh units saturate and ReLU outputs explode.	Either saturation (gradient ≈ 0) or numerical overflow (loss becomes NaN).

3. Xavier (Glorot) and He initialisation

The design goal is to keep the **variance of the activations and of the gradients approximately constant across layers**. Analysing $\text{Var}(z) = n_{in} \cdot \text{Var}(w) \cdot \text{Var}(x)$ gives the required variance for the weights.

```
Xavier / Glorot (for tanh, sigmoid):
    Var(w) = 1 / n_in     or     2 / (n_in + n_out)
    Uniform form: w ~ U[-√(6/(n_in+n_out)), +√(6/(n_in+n_out))]

He / Kaiming (for ReLU and its variants):
    Var(w) = 2 / n_in      →     w ~ N(0, √(2/n_in))
```

The factor of 2 in He initialisation compensates for the fact that ReLU zeroes out approximately half of its inputs, halving the variance of the layer's output; doubling the weight variance restores it.

Numerical example for a layer with $n_{in} = 256$ and $n_{out} = 128$:

```
Xavier (1/n_in)       : σ = √(1/256)          = 0.0625
Xavier (normalised)   : σ = √(2/(256+128))    = 0.0722
He (2/n_in)           : σ = √(2/256)          = 0.0884
```

He initialisation gives noticeably wider weights, which is exactly what a ReLU layer needs to avoid a progressive shrinkage of activations through depth.

Scheme	Variance	Intended activation	Typical use
Xavier / Glorot	1/n_in or 2/(n_in+n_out)	tanh, sigmoid, softmax	Shallow networks, RNNs, output layers
He / Kaiming	2/n_in	ReLU, Leaky ReLU, ELU	**Default for modern deep CNNs and MLPs**
LeCun	1/n_in	SELU	Self-normalising networks
Orthogonal	orthonormal rows	any	Deep RNNs, where repeated multiplication demands norm preservation

Biases are normally initialised to 0 (they do not cause symmetry problems, because the weights already break it). A small positive bias such as 0.01 is occasionally used with ReLU to reduce the chance of units starting out dead.

PART B — OVERFITTING, UNDERFITTING AND REGULARISATION

4. Overfitting versus underfitting

Aspect	Underfitting (high bias)	Overfitting (high variance)
Symptom	Training error high, validation error high and similar	Training error very low, validation error much higher — a large and growing gap
Cause	Model too simple, too few layers/neurons, over-regularised, trained too briefly, or learning rate poorly set	Model too complex relative to the data, too few training samples, trained too long, noisy or unrepresentative data
What the model learned	Not even the underlying pattern	The pattern **plus** the noise and idiosyncrasies of the training set
Remedy	Increase capacity (more layers/neurons), train longer, reduce regularisation, engineer better features	Add regularisation, gather more data, use data augmentation, reduce capacity, apply early stopping

The goal is the **bias–variance trade-off**: enough capacity to capture the true structure, with enough constraint that the model does not memorise noise. The practical signature of overfitting is a validation-loss curve that reaches a minimum and then begins to rise while the training loss keeps falling.

5. Regularisation techniques

Technique	How it works	Effect
L2 regularisation (weight decay)	Adds $\lambda\Sigma w^2$ to the loss, so the gradient gains a term $2\lambda w$ that continually shrinks weights towards zero.	Discourages any single weight from becoming large, producing smoother functions and better generalisation. The most common default.
L1 regularisation	Adds $\lambda\Sigma\|w\|$ to the loss.	Drives many weights to exactly zero, producing a **sparse** model that performs implicit feature selection.

Technique	How it works	Effect
Dropout	During training, each neuron is deactivated independently with probability p (typically 0.2–0.5). At test time all neurons are used, with activations scaled appropriately.	Prevents co-adaptation of neurons and approximates averaging over an exponential ensemble of thinned networks. Very effective in fully connected layers.
Early stopping	Monitor validation loss each epoch and halt training when it stops improving for a set number of epochs (patience), restoring the best checkpoint.	Prevents the model from continuing into the regime where it begins memorising the training set. Costs nothing and is almost always worth using.
Data augmentation	Generate new training samples by applying label-preserving transformations — for images: flips, crops, rotations, colour jitter, mixup.	Effectively enlarges the dataset and teaches invariances, which is often the single most effective remedy for overfitting.
Batch normalisation	Normalises each layer's pre-activations per mini-batch, then rescales with learned parameters γ and β.	Stabilises and accelerates training, allows higher learning rates, and provides a mild regularising effect through mini-batch noise.
Reducing model capacity	Use fewer layers or fewer neurons per layer.	Directly limits the model's ability to memorise, though at the risk of tipping into underfitting.

6. Practical workflow

1. Split the data into training, validation and test sets, and never tune on the test set.
2. Initialise with He (for ReLU networks) or Xavier (for tanh/sigmoid networks); set biases to zero.
3. Start with a model large enough to **overfit** the training data — this confirms the architecture has sufficient capacity and that the pipeline works.
4. Then add regularisation progressively — weight decay, dropout, augmentation — until the validation loss is minimised.
5. Use early stopping with a patience of a few epochs and restore the best-performing checkpoint.
6. Evaluate once on the held-out test set to obtain an unbiased estimate of generalisation performance.